QuillAudits

AUDIT REPORT

July 2026

For

SPACEWAY TOKENS

SPWAY

Table of Content

Executive Summary 03

Number of Security Issues per Severity 05

Summary of Issues 06

Checked Vulnerabilities 07

Techniques and Methods 09

Types of Severity 11

Types of Issues 12

Severity Matrix 13

Low Severity Issues 14

1. Missing zero-address validation in initialize 14

Informational Issues 15

2. initialize declared public instead of external 15

3. Redundant zero-address check in _authorizeUpgrade 16

Functional Tests 17

Threat Model 18

Automated Tests 26

Closing Summary & Disclaimer 27

Executive Summary

Project Name	SpaceWay
Protocol Type	ERC20
Project URL	https://spacewaytoken.com/
Overview	The SpaceWay contract is an upgradeable ERC-20 token built using OpenZeppelin's UUPS proxy pattern. It extends the standard ERC-20 functionality with EIP-2612 permit support for gasless approvals and implements role-based access control to manage administrative privileges and upgrade authorization. During initialization, the contract mints a fixed supply of 2 billion SPWAY tokens to a designated recipient while assigning separate administrator and upgrader roles. The upgrade mechanism is restricted to accounts holding the UPGRADER_ROLE, with an additional validation preventing upgrades to the zero address, ensuring controlled and secure implementation upgrades.
Audit Scope	The scope of this Audit was to analyze the SpaceWay Smart Contracts for quality, security, and correctness.
Source Code link	https://etherscan.io/address/0x02d7b5823420c6c964e8203942e073ce7d320b87#code
Contracts in Scope	Spway.sol
Language	Solidity
Blockchain	EVM
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	6th July 2026
Updated Code Received	7th July 2026
Review 2	7th July 2026
Fixed In	0xada288a38c0d54132746f21063c7c79dc4bb8f86

Verify the Authenticity of Report on QuillAudits Leaderboard:

https://www.quillaudits.com/leaderboard

Number of Issues per Severity

Critical	0 (0%)
High	0 (0%)
Medium	0 (0%)
Low	1 (33.33%)
Informational	2 (66.67%)

Severity

Issues	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	0	0	0
Partially Resolved	0	0	0	0	0
Resolved	0	0	0	**1**	**2**

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Missing zero-address validation in initialize	Low	Resolved
2	initialize declared public instead of external	Informational	Resolved
3	Redundant zero-address check in _authorizeUpgrade	Informational	Resolved

Checked Vulnerabilities

- Access Management
- Arbitrary write to storage
- Centralization of control
- Ether theft
- Improper or missing events
- Logical issues and flaws
- Arithmetic Computations Correctness
- Race conditions/front running
- SWC Registry
- Re-entrancy
- Timestamp Dependence
- Gas Limit and Loops
- Exception Disorder
- Gasless Send
- Use of tx.origin
- Malicious libraries
- Compiler version not fixed
- Address hardcoded
- Divide before multiply
- Integer overflow/underflow
- ERC's conformance
- Dangerous strict equalities
- Tautology or contradiction
- Return values of low-level calls

- Missing Zero Address Validation
- Private modifier
- Revert/require functions
- Multiple Sends
- Using suicide
- Using delegatecall
- Upgradeable safety
- Using throw
- Using inline assembly
- Style guide violation
- Unsafe type inference
- Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- Medium - only a conditionally incentivized attack vector, but still relatively likely.
- Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Low Severity Issues

Missing zero-address validation in initialize

Resolved

Path

Spway.sol

Function Name

`initialize()`

Description

initialize accepts recipient, defaultAdmin, and upgrader and passes them straight into _mint and _grantRole without any non-zero validation. _mint inherits OZ's ERC20._update guard, so a zero recipient reverts on its own and is implicitly protected. The two role parameters have no such guard: _grantRole(DEFAULT_ADMIN_ROLE, address(0)) and _grantRole(UPGRADER_ROLE, address(0)) both succeed silently. Because the function carries the initializer modifier, it executes exactly once with no possibility of correction.

Impact

The severe case is defaultAdmin == address(0). DEFAULT_ADMIN_ROLE is the admin of every other role (including UPGRADER_ROLE), so if it's assigned to the zero address, no account can ever call grantRole / revokeRole. Role administration is permanently bricked the token still transfers, but governance is frozen for the life of the proxy. The upgrader == address(0) case is less severe: upgrades are disabled, but a valid defaultAdmin can still grant UPGRADER_ROLE to a fresh address, so it's recoverable. The genuinely unrecoverable failure mode is a zero admin.

Likelihood:

Low. It requires operator error at deploy time and cannot be triggered post-deployment.

Recommendation

Add explicit non-zero checks in initialize for defaultAdmin and upgrader. The recipient check is redundant given _mint's guard, but including it for symmetry is harmless.

Informational Severity Issues

initialize declared public instead of external

Resolved

Path

Spway.sol

Function Name

`initialize()`

Description

initialize is declared public but is never called internally. Functions invoked only from outside the contract should be external, which is marginally more gas-efficient for calldata handling and communicates intent more precisely.

Recommendation

Change the signature to external

initialize declared public instead of external

Resolved

Path

Spway.sol

Function Name

`_authorizeUpgrade`

Description

_authorizeUpgrade contains require(newImplementation != address(0), ...). By the time this hook runs, the UUPS upgrade path in ERC1967Utils.upgradeToAndCall has already validated the target: it reverts on a non-contract address (code.length == 0, which necessarily excludes address(0)) and additionally verifies the target's proxiableUUID. The check is therefore dead and it can never be the operative revert reason for a real upgrade attempt.

Recommendation

Remove the redundant check

Functional Tests

Some of the tests performed are mentioned below:

- It should mint exactly 2_000_000_000 * 10**18 tokens to recipient on initialize.
- It should revert on any second call to initialize.
- It should set the allowance via permit when given a valid owner signature within the deadline.
- It should allow an account to renounce its own role via renounceRole, and revert if it tries to renounce on behalf of another account.
- It should increment the owner's nonce after a successful permit.

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
ERC20Upgradeable (OpenZeppelin)	Library / Base Contracts	Provides the core ERC20 token logic (balances, transfers, allowances) for SPWAY	Correct, standard-compliant ERC20 accounting with no hidden mint/burn paths	If library has a bug, balance/allowance accounting is corrupted
ERC20Upgradeable (OpenZeppelin)	Library / Base Contracts	Adds EIP-2612 gasless approvals (permit) with EIP-712 signatures and per-owner nonces	Correct signature recovery, domain separation, and nonce management	Signature replay or malleability if library is flawed; permit front-running
AccessControlUpgradeable (OpenZeppelin)	Library / Base Contracts	Role-based access control for DEFAULT_ADMIN_ROLE and UPGRADER_ROLE	Roles are correctly enforced; role admins are trusted and competent	Compromised/malicious admin can grant roles; over-privileged UPGRADER_ROLE

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Initializable (OpenZeppelin)	Library/ Base Contract	Guards one-time initialization for the proxy pattern; disables re-initialization	initializer/ reinitializer modifiers correctly prevent double init	Uninitialized-implementation or re-init bugs if misused
UUPSUpgradeable (OpenZeppelin)	Library/ Base Contract	Enables in-place logic upgrades via upgradeToAndCall, gated by _authorizeUpgrade	Upgrade authorization is correctly enforced; storage layout preserved across upgrades	Malicious/ incorrect upgrade can replace all logic; storage-layout collision
ERC1967 Proxy (deployment-time)	External Proxy Contract	Delegatecalls into the SpaceWay implementation and stores all persistent state	Proxy is deployed correctly and initialized atomically with the implementation	Front-run initialization; wrong implementation slot; delegatecall context

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

LienNFT.SOL

Contract Name	Function	Category
DefinixToken.sol	constructor()	**What this function can do** Mint the full token supply and assign it to msg.sender **What this function cannot/should not do** Mint again after deployment, assign tokens to arbitrary addresses **Main invariant(s)** Total supply is fixed at deployment **Access level** Implicit restriction (once at deployment)
SpaceWay.sol	constructor()	**What this function can do** Disable initializers on the implementation contract so the logic contract can never be initialized directly **What this function cannot/should not do** Set proxy state, mint tokens, or be re-run after deployment **Main invariant(s)** The implementation contract is permanently uninitialized; only proxies hold usable state **Access level** Implicit restriction (runs once at implementation deployment)

Contract Name	Function	Category
SpaceWay.sol	initialize(recipient, defaultAdmin, upgrader)	**What this function can do** Set token name/symbol ("SpaceWay"/"SPWAY") and EIP-712 permit domain, mint the fixed 2,000,000,000e18 supply to recipient, grant DEFAULT_ADMIN_ROLE to defaultAdmin and UPGRADER_ROLE to upgrader **What this function cannot/should not do** Be called more than once, mint additional supply afterwards, be called with zero addresses (not validated) **Main invariant(s)** Runs exactly once per proxy; total supply fixed at 2B; entire supply held by recipient at genesis **Access level** Public, guarded by initializer (one-time); should be called atomically with proxy deployment to prevent front-running
SpaceWay.sol	upgradeToAndCall(newImplementation, data)	**What this function can do** Replace the proxy's implementation contract and optionally execute a call on the new implementation (gated by _authorizeUpgrade) **What this function cannot/should not do** Be called by a non-UPGRADER_ROLE address, or set the implementation to the zero address (explicitly reverted) **Main invariant(s)** Only UPGRADER_ROLE can upgrade; new implementation is non-zero **Access level** Restricted onlyRole(UPGRADER_ROLE) via _authorizeUpgrade

Contract Name	Function	Category
SpaceWay.sol	transfer(to, amount)	**What this function can do** Move SPWAY from msg.sender to to, decreasing sender balance and increasing recipient balance **What this function cannot/should not do** Transfer more than msg.sender's balance, mint or create tokens **Main invariant(s)** Sum of balances is conserved; total supply unchanged **Access level** Public
SpaceWay.sol	transferFrom(from, to, amount)	**What this function can do** Move SPWAY from from to to using msg.sender's allowance, decreasing the allowance accordingly **What this function cannot/should not do** Transfer without sufficient allowance/ balance, mint tokens **Main invariant(s)** Allowance is respected and decreased; total supply unchanged **Access level** Public (requires prior approval/permit)

Contract Name	Function	Category
SpaceWay.sol	approve(spender, amount)	**What this function can do** Set spender's allowance over msg.sender's SPWAY to amount **What this function cannot/should not do** Move tokens, or set allowance on behalf of another owner **Main invariant(s)** Allowance mapping reflects the last approved value **Access level** Public
SpaceWay.sol	permit(owner, spender, value, deadline, v, r, s)	**What this function can do** Set spender's allowance over owner's SPWAY via an off-chain EIP-712 signature, consuming one nonce **What this function cannot/should not do** Succeed after deadline, reuse a nonce, or accept an invalid/mismatched-domain signature **Main invariant(s)** Each signature is single-use (nonce increments); signature binds owner/spender/value/deadline **Access level** Public (anyone may submit a valid signature; approval authority is the signer)

Contract Name	Function	Category
SpaceWay.sol	grantRole(role, account)	**What this function can do** Grant role (e.g., UPGRADER_ROLE) to account **What this function cannot/should not do** Grant a role without being the role's admin **Main invariant(s)** Only the role's admin (DEFAULT_ADMIN_ROLE by default) can grant it **Access level** Restricted onlyRole(getRoleAdmin(role))
SpaceWay.sol	revokeRole(role, account)	**What this function can do** Remove role from account **What this function cannot/should not do** Revoke a role without being the role's admin **Main invariant(s)** Only the role's admin can revoke it **Access level** Restricted onlyRole(getRoleAdmin(role))
SpaceWay.sol	renounceRole(role, callerConfirmation)	**What this function can do** Allow an account to voluntarily give up its own role **What this function cannot/should not do** Renounce a role on behalf of another account (callerConfirmation must equal msg.sender) **Main invariant(s)** Only the role holder can renounce its own role

Contract Name	Function	Category
		Access level Public (self-only; renouncing DEFAULT_ADMIN_ROLE can permanently orphan admin control)

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
SpaceWay.sol (proxy)	ERC20 (SPWAY)	The full SPWAY supply is tracked in the internal balance ledger; the contract address itself custodies no external assets by default

3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
SpaceWay.sol	initialize	–	SPWAY (2B minted)	Deployer (once)	Entire supply minted to a single recipient; centralization / trusted distribution; front-run risk if not atomic

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
SpaceWay.sol	transfer	SPWAY	SPWAY	Public	Standard transfer; no fee-on-transfer or rebasing
SpaceWay.sol	transferFrom	SPWAY	SPWAY	Public (approved)	Requires allowance; approval race condition (classic ERC20 allowance issue)
SpaceWay.sol	permit + transferFrom	SPWAY	SPWAY	Public (signer-authorized)	Gasless approval; signature front-running / griefing on nonce
SpaceWay.sol	upgradeToAndCall	-	-	UPGRADER_ROLE	Upgrade can arbitrarily rewrite token/asset logic and access all state; single most privileged path

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Closing Summary

In this report, we have considered the security of SpaceWay. We performed our audit according to the procedure described above.

1 Low and 2 Informational severity issues were found,which Spaceway team Fixed Successfully.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With seven years of expertise, we've secured over 1400 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey

AUDIT REPORT

July 2026

For

SPACEWAY TOKENS

SPWAY

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com